EXPERIMENT 10

Ensemble Learning – Random Forest and Boosting (Cell by Cell)

Aim: To implement and compare Logistic Regression, Random Forest, and Gradient Boosting classifiers on the Breast Cancer dataset using classification metrics and visualizations.

Dataset: Built-in Breast Cancer Wisconsin dataset from scikit-learn.

Cell 1: Import libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, precision_score
from sklearn.metrics import recall_score, f1_score, confusion_matrix
from sklearn.pipeline import make_pipeline
```

Cell 2: Load dataset

```
data = load_breast_cancer()

X = data.data
y = data.target
```

Cell 3: Convert into DataFrame

```
df = pd.DataFrame(X, columns=data.feature_names)
df['target'] = y

print("First Five Records:")
print(df.head())
```

Cell 4: Check dataset information

```
print("\nDataset Information:")
print(df.info())
```

Cell 5: Check missing values

```
print("\nMissing Values:")
print(df.isnull().sum().sum())
```

Cell 6: Display class distribution

```
print("\nClass Distribution:")
print(df['target'].value_counts())
```

Cell 7: Separate features and target

```
X = df.drop('target', axis=1)
y = df['target']
```

Cell 8: Split dataset into training and testing

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Cell 9: Create models

```
models = {
    "Logistic Regression": make_pipeline(
        StandardScaler(),
        LogisticRegression(max_iter=5000)
    ),

    "Random Forest": RandomForestClassifier(
        n_estimators=100, random_state=42
    ),

    "Gradient Boosting": GradientBoostingClassifier(
        n_estimators=100, random_state=42
    )
}
```

Cell 10: Train and evaluate models

```
results = []
conf_matrices = {}

for name, model in models.items():

    # Train model
    model.fit(X_train, y_train)

    # Prediction
    y_pred = model.predict(X_test)

    # Evaluation metrics
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    results.append([
        name, accuracy, precision, recall, f1
    ])

    # Confusion matrix
    cm = confusion_matrix(y_test, y_pred)
    conf_matrices[name] = cm

    print("\n", name)
    print("Accuracy:", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall:", round(recall, 4))
    print("F1 Score:", round(f1, 4))
```

Cell 11: Compare all models

```
comparison = pd.DataFrame(
    results,
    columns=[
        "Model", "Accuracy", "Precision",
        "Recall", "F1 Score"
    ]
)

print("\nModel Performance Comparison:")
print(comparison.round(4))
```

Cell 12: Plot confusion matrices

```
plt.figure(figsize=(15, 4))

for i, (name, cm) in enumerate(conf_matrices.items()):

    plt.subplot(1, 3, i + 1)

    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        cmap='Blues',
        xticklabels=data.target_names,
        yticklabels=data.target_names
    )

    plt.title(name)
    plt.xlabel("Predicted")
    plt.ylabel("Actual")

plt.tight_layout()
plt.show()
```

Cell 13: Compare model accuracy

```
plt.figure(figsize=(8, 5))

plt.bar(
    comparison['Model'],
    comparison['Accuracy']
)

plt.title("Accuracy Comparison of Models")
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.ylim(0, 1.1)
plt.show()
```

Cell 14: Compare evaluation metrics

```
comparison.set_index('Model').plot(
    kind='bar',
    figsize=(10, 5)
)

plt.title("Performance Comparison of Models")
plt.ylabel("Score")
plt.ylim(0, 1.1)
plt.xticks(rotation=0)
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()
```

Cell 15: Final Conclusion

```
print("\nConclusion:")
print("Ensemble Learning experiment completed successfully.")

best_model = comparison.loc[
    comparison['Accuracy'].idxmax()
]

print("Best Model:", best_model['Model'])
print("Best Accuracy:", round(best_model['Accuracy'], 4))
```

Note: Run the cells in order, from Cell 1 to Cell 15. Model metrics, confusion matrices, plots, and the best model are generated when the code is executed.